Using the STU-SDK for Linux on Fedora–27 64 bit

The Linux STU-SDK uses libusb and OpenSSL for commiuncation for the STU devices. Before you are able to run the sample applications provided with the STU-SDK, you will need to perform a number of steps.

Installing the build environment

Firstly, you will need to install a build enviroment. To do this, type the following command into a terminal window, entering the root password and selecting ‘y’ when prompted:

```
sudo dnf update && sudo dnf install @development-tools && sudo dnf install gcc-g++
```

Install libusb

The STU-SDK uses libusb for communication over USB to STU devices. To install the development libraries for libusb, enter the following command in a terminal window, entering the root password and selecting ‘y’ when prompted:

```
sudo dnf install libusb-devel.x86_64
```

Install OpenSSL

For secured communications, the STU-SDK uses the OpenSSL library. To install the development libraries for OpenSSL, enter the following command in a terminal window, entering the root password and selecting ‘y’ when prompted:

```
mkdir -p /tmp/openssl_scratch && cd /tmp/openssl_scratch
wget https://www.openssl.org/source/openssl-1.0.2n.tar.gz
tar -zxvf openssl-1.0.2n.tar.gz && cd openssl-1.0.2n
./config --prefix=/usr && make && sudo make install
```

Setup uDev rules

In order to allow connection to the STU device from user space, the udev rules must be configured correctly. Enter the following command in terminal to configure the rules, entering your password when prompted:

```
sudo su -
cat <<EOF >> /etc/udev/rules.d/60-stu.rules
SUBSYSTEM=="usb",ATTR{idVendor}=="056a",MODE:="0666"
SUBSYSTEM=="usb_device",ATTR{idVendor}=="056a",MODE:="0666"
EOF
udevadm control --reload-rules
logout
```

Note: You will need to unplug and re-connect your STU device for the updated rules to take effect.

Download the STU-SDK

Nagivate to http://developer.wacom.com/ and select login at the top right of the page. Once you have logged in with your Wacom ID, select Resources->Downloads from the top menu, then select 'Wacom Device Kit'. from this page, select 'Download STU-SDK for Linux (current version at the time of writing is 2.13.6) and Downlad this to your Desktop or other known location. Unzip the distribution:

```
[joss@localhost Desktop]$ unzip STU-SDK-Linux-2.13.6.zip
Archive:  STU-SDK-Linux-2.13.6.zip
   creating: STU-SDK-Linux-2.13.6/
   creating: STU-SDK-Linux-2.13.6/documentation/
  inflating: STU-SDK-Linux-2.13.6/documentation/Linux-STU-SDK-Guide.pdf
   creating: STU-SDK-Linux-2.13.6/sdk/
  inflating: STU-SDK-LinuxYou will now need to unplug and re-connect your STU device for the u
pdated rules to take effect.-2.13.6/sdk/.DS_Store
  inflating: STU-SDK-Linux-2.13.6/sdk/Wacom-STU-SDK-2.13.6.tar.bz2
  inflating: STU-SDK-Linux-2.13.6/README.txt
[joss@localhost Desktop]$
```

Then change to the sdk directory and extract the SDK:

```
[joss@localhost Desktop]$ cd STU-SDK-Linux-2.13.6/sdk
[joss@localhost sdk]$ bzip2 -d Wacom-STU-SDK-2.13.6.tar.bz2 && tar -xf Wacom-STU-SDK-2.13.6.ta
r
[joss@localhost sdk]$
```

You are now ready to build and run the C++ Samples.

Building and running the C++ samples

First, you will need to build the C++ library. In the current distrubution there is a missing include in ../include/WacomGSS/STU/Interface.hpp, so you will need to add the following include declaration to this file:

```
#include <functional>
```

There are also a few modifications that are required to the make file. Replace the contents of the `gcc46.mak` file under the `Wacom-STU-SDK/cpp/build/` with the following data:

```
# gcc46.mak
# Copyright (c) 2015 Wacom Company Limited
# 2012-03-09 mholden
#
# tested on: g++ (GCC) 4.6.2 20111027 (Red Hat 4.6.2-1)  --  3.2.9-1.fc16.x86_64
#
# Dependencies:
#  openssl
#  libusb-1.0
#

ifndef MACHTYPE
$(error MACHTYPE not defined)
endif

OutDir=Linux/$(MACHTYPE)/
IntDir=Linux/$(MACHTYPE)/i/

ifndef SrcDir
SrcDir=../src/
endif

ifndef SamplesDir
SamplesDir=../samples/
endif

ifndef INCLUDEPATH
INCLUDEPATH+= -I../include
endif

INCLUDEPATH+= -I/usr/include/libusb-1.0

CPP=g++
CPPFLAGS=-c -Wall -Wno-unknown-pragmas $(INCLUDEPATH) -std=c++11 -fPIC -shared -fvisibility=hi
dden -fvisibility-inlines-hidden -g -ggdb -O0

ifeq ($(MACHTYPE),i686)
LIBPATH=/lib/
LIBBOOSTPATH=/usr/lib/
CPPFLAGS+= -m32
endif
ifeq ($(MACHTYPE),x86_64)
LIBPATH=/lib64/
LIBBOOSTPATH=/usr/lib/x86_64-linux-gnu/
endif


ifndef LIBPATH
$(error LIBPATH not defined)
endif

#       ------------------------------------------------------------------------

build   :       $(OutDir) $(IntDir)     $(OutDir)WacomGSS.a $(OutDir)getUsbDevices $(OutDir)si
mpleInterface $(OutDir)simpleTablet $(OutDir)query

.PHONY: build clean rebuild

clean   :
```

```
    -rm -Rf $(OutDir)

rebuild :       clean   build

#       ------------------------------------------------------------------------

WacomGSS_Linux=\
    $(IntDir)libusb.a   \
    $(IntDir)libusbHelper.a

WacomGSS_utility=\
    $(IntDir)setThreadName.a    \
    $(IntDir)enumUsbDevices_libusb.a

WacomGSS_STU=\
    $(IntDir)Error.a    \
    $(IntDir)getUsbDevices.a    \
    $(IntDir)getTlsDevices.a    \
    $(IntDir)getUsbDevices_libusb.a     \
    $(IntDir)getTlsDevices_libusb.a     \
    $(IntDir)Interface.a        \
    $(IntDir)InterfaceImpl.a    \
    $(IntDir)InterfaceQueue.a   \
    $(IntDir)Protocol.a \
    $(IntDir)ProtocolHelper.a   \
    $(IntDir)ReportHandler.a    \
    $(IntDir)SerialInterface.a  \
    $(IntDir)SerialProtocol.a   \
    $(IntDir)Tablet.a   \
    $(IntDir)TlsInterface.a     \
    $(IntDir)TlsInterfaceImpl.a \
    $(IntDir)TlsInterface_Debug.a       \
    $(IntDir)TlsProtocol.a      \
    $(IntDir)TlsProtocolOOB.a   \
    $(IntDir)UsbInterface.a

WacomGSS_OpenSSL=\
    $(IntDir)OpenSSL_Loader.a           \
    $(IntDir)OpenSSL_eay.a              \
    $(IntDir)OpenSSL_ssl.a              \
    $(IntDir)Tablet_OpenSSL.a

WacomGSS_all=\
    $(WacomGSS_STU)     \
    $(WacomGSS_OpenSSL) \
    $(WacomGSS_Linux)   \
    $(WacomGSS_utility)

#       ------------------------------------------------------------------------

$(OutDir)       :
    mkdir -p $(OutDir)

$(IntDir)       :
    mkdir -p $(IntDir)

#       ------------------------------------------------------------------------

$(OutDir)WacomGSS.a : $(WacomGSS_all)
    ar r $@ $^

#       ------------------------------------------------------------------------

$(IntDir)setThreadName.a        :       $(SrcDir)utility/cpp/Linux/setThreadName.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<
```

```
$(IntDir)enumUsbDevices_libusb.a        :       $(SrcDir)utility/cpp/Linux/enumUsbDevices_libu
sb.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)OpenSSL_applink.a      :       $(SrcDir)utility/cpp/OpenSSL_applink.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)OpenSSL_Loader.a       :       $(SrcDir)utility/cpp/Linux/OpenSSL_Loader.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)OpenSSL_eay.a  :       $(SrcDir)utility/cpp/OpenSSL_eay.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)OpenSSL_ssl.a  :       $(SrcDir)utility/cpp/OpenSSL_ssl.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)libusb.a       :       $(SrcDir)Linux/cpp/libusb.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)libusbHelper.a :       $(SrcDir)Linux/cpp/libusbHelper.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)Error.a        :       $(SrcDir)STU/cpp/Error.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)getUsbDevices.a        :       $(SrcDir)STU/cpp/getUsbDevices.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)getTlsDevices.a        :       $(SrcDir)STU/cpp/getTlsDevices.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)Interface.a    :       $(SrcDir)STU/cpp/Interface.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)InterfaceImpl.a        :       $(SrcDir)STU/cpp/InterfaceImpl.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)InterfaceQueue.a       :       $(SrcDir)STU/cpp/InterfaceQueue.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)Protocol.a     :       $(SrcDir)STU/cpp/Protocol.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)ProtocolHelper.a       :       $(SrcDir)STU/cpp/ProtocolHelper.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)ReportHandler.a        :       $(SrcDir)STU/cpp/ReportHandler.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)SerialProtocol.a       :       $(SrcDir)STU/cpp/SerialProtocol.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)Tablet.a       :       $(SrcDir)STU/cpp/Tablet.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)Tablet_OpenSSL.a       :       $(SrcDir)STU/cpp/Tablet_OpenSSL.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)SerialInterface.a      :       $(SrcDir)STU/cpp/Linux/SerialInterface.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)UsbInterface.a :       $(SrcDir)STU/cpp/Linux/UsbInterface.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<
```

```
$(IntDir)TlsInterface.a :       $(SrcDir)STU/cpp/Linux/TlsInterface.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)TlsInterfaceImpl.a     :       $(SrcDir)STU/cpp/TlsInterfaceImpl.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)TlsInterface_Debug.a   :       $(SrcDir)STU/cpp/TlsInterface_Debug.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)TlsProtocol.a  :       $(SrcDir)STU/cpp/TlsProtocol.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)TlsProtocolOOB.a       :       $(SrcDir)STU/cpp/TlsProtocolOOB.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)getUsbDevices_libusb.a :       $(SrcDir)STU/cpp/Linux/getUsbDevices_libusb.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(IntDir)getTlsDevices_libusb.a :       $(SrcDir)STU/cpp/Linux/getTlsDevices_libusb.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

$(OutDir)getUsbDevices  :       $(IntDir)sample_getUsbDevices.a $(OutDir)WacomGSS.a
    g++ -o $@ $^ -lusb-1.0 $(BOOSTLIBS)

$(IntDir)sample_getUsbDevices.a :       $(SamplesDir)getUsbDevices.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

#       ---------------------------------------------------------------------------

$(OutDir)simpleInterface        :       $(IntDir)sample_simpleInterface.a $(OutDir)WacomGSS.a
    g++ -o $@ $^ -lusb-1.0 -lpthread -lrt -ldl -lssl -lcrypto  $(BOOSTLIBS)

$(IntDir)sample_simpleInterface.a       :       $(SamplesDir)simpleInterface.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

#       ---------------------------------------------------------------------------

$(OutDir)simpleTablet   :       $(IntDir)sample_simpleTablet.a $(OutDir)WacomGSS.a
    g++ -o $@ $^   -lusb-1.0 -lssl -lcrypto  -ldl -lpthread -lrt $(BOOSTLIBS)

$(IntDir)sample_simpleTablet.a  :       $(SamplesDir)simpleTablet.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

#       ---------------------------------------------------------------------------

$(OutDir)query  :       $(IntDir)sample_query.a $(OutDir)WacomGSS.a
    g++ -o $@ $^  -lusb-1.0 -lssl -lcrypto  -ldl -lpthread -lrt $(BOOSTLIBS)

$(IntDir)sample_query.a :       $(SamplesDir)query.cpp
    $(CPP) $(CPPFLAGS) -o $@ $<

#       ---------------------------------------------------------------------------
```

Finally, change to the cpp build directory, then run make:

```
[joss@localhost sdk]$ cd Wacom-STU-SDK/cpp/build/
[joss@localhost build]$ make MACHTYPE=x86_64 -f gcc46.mak
```

You should now be able to run the C++ samples under `Linux/x86_64` directory of the build directory, e.g.:

```
[joss@localhost build]$ cd Linux/x86_64/
[joss@localhost x86_64]$ ls
getUsbDevices  i  query  simpleInterface  simpleTablet  WacomGSS.a
[joss@localhost x86_64]$ ./getUsbDevices
056a:00a2:0103
[joss@localhost x86_64]$ ./simpleInterface
STU simpleInterface sample

No TLS devices found
Connecting to first device found: 056a:00a2:0103
Connecting...
Connected!
getInformation()... modelName=STU-300 firmware=01.03.0
getReportRate()... NOT SUPPORTED
getPenDataOptionMode()... NOT SUPPORTED
setClearScreen()... ok!
setInkingMode(On)... ok!
(use stylus, press CTRL-C to quit)

rdy  sw     x     y  pres
  1   0     0  7559   957
  1   0     0  7557   962
  1   0     0  7556   971
  1   0     0  7554   981
  1   0     0  7554   995
  1   0     0  7553  1008
  1   0     0  7555  1022
  1   0     0  7558  1039
  1   0     0  7567  1055

...
```